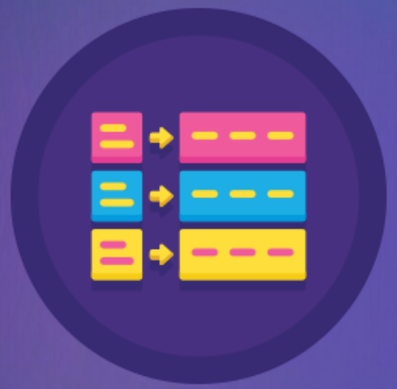
1eron.berisha@universitetiaab.com

BAZA E TË DHËNAVE



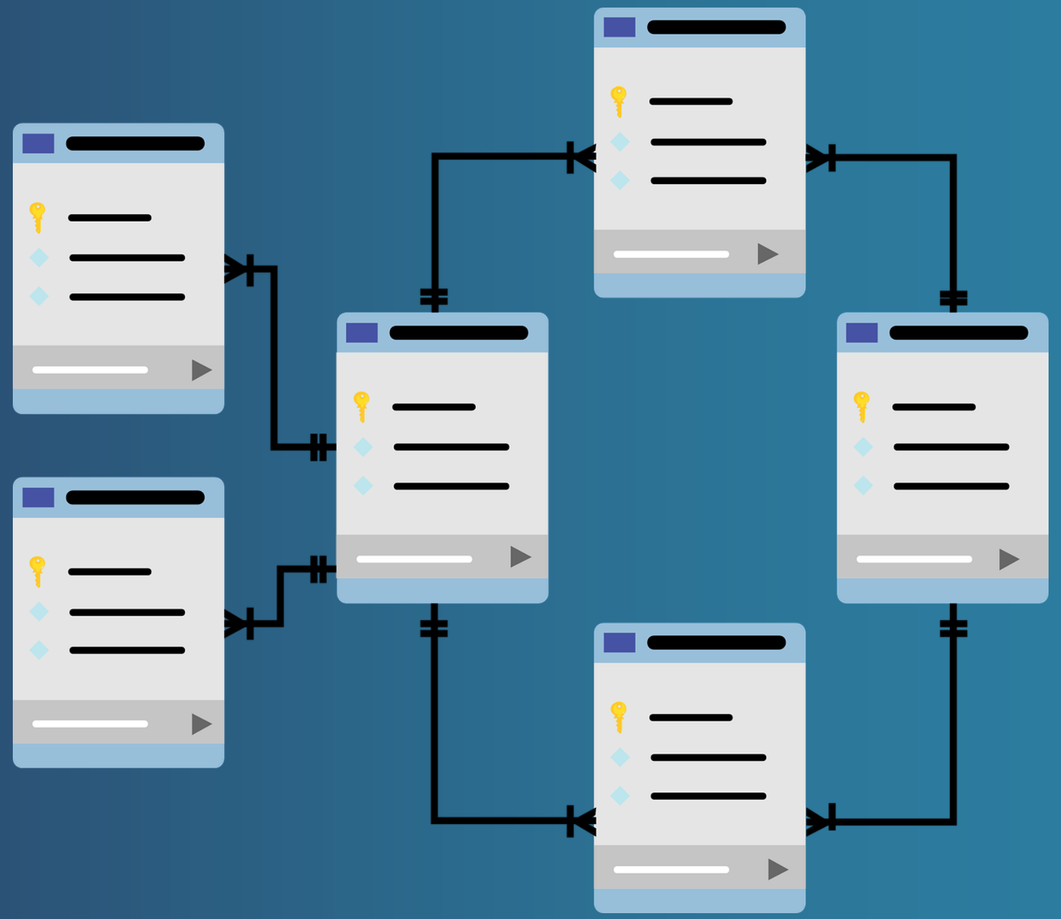




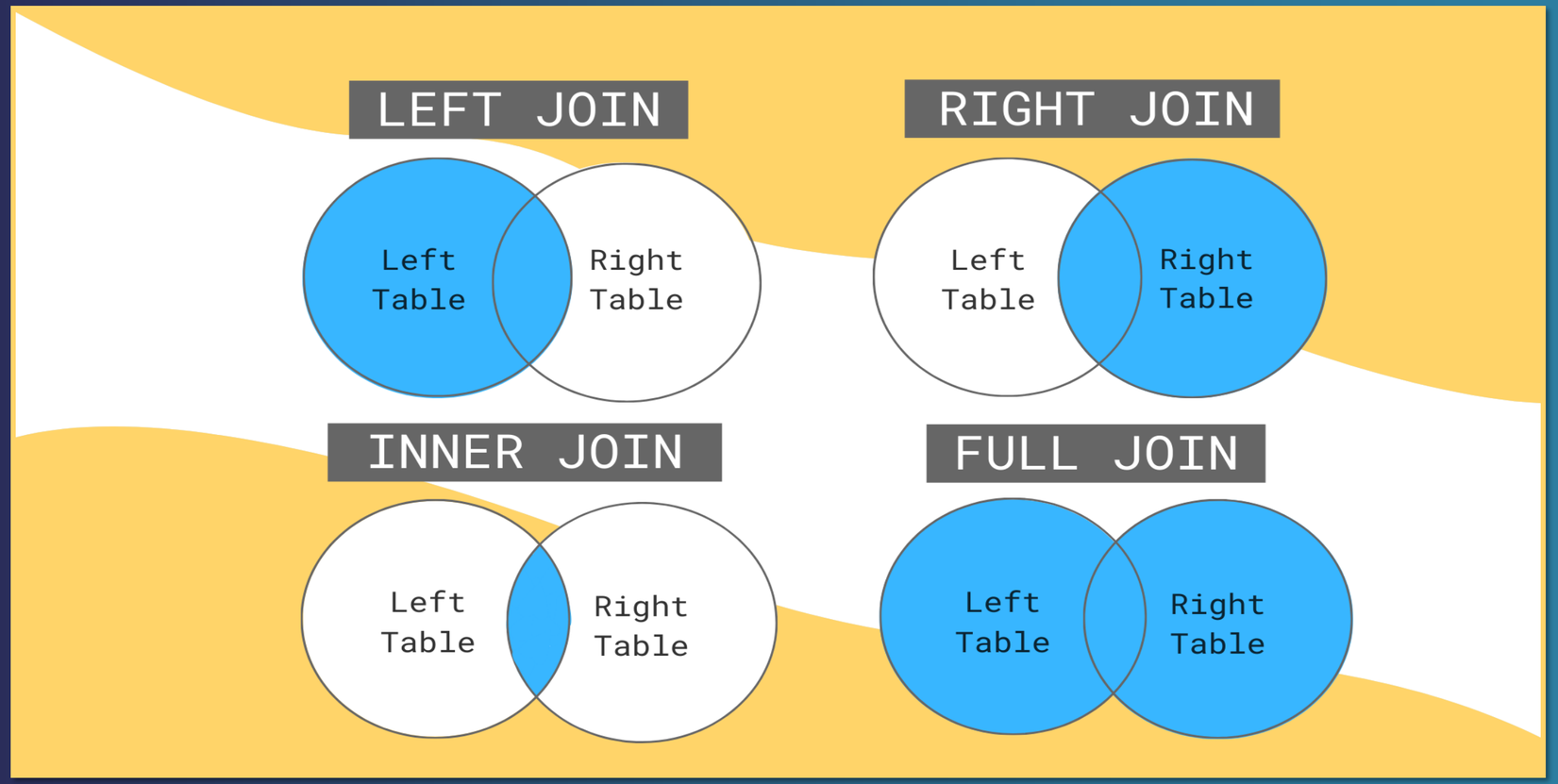
JAVA X

JOIN

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- FULL JOIN

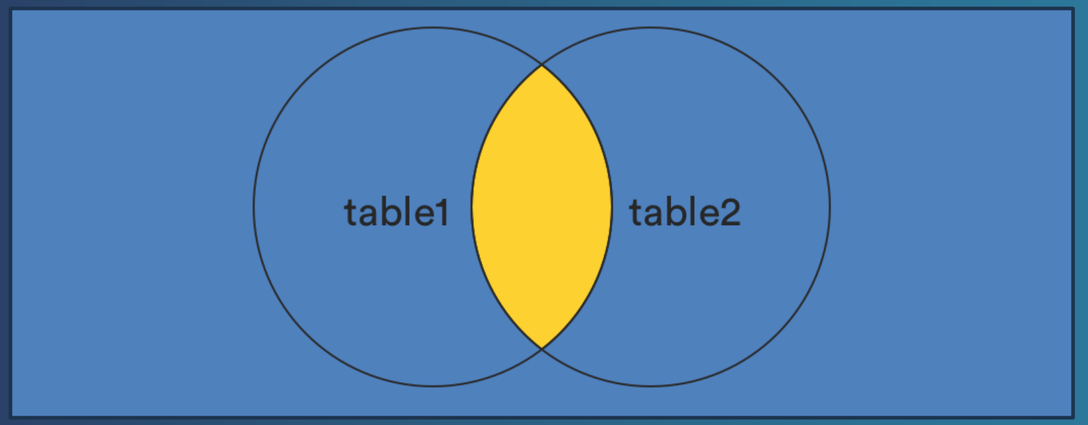


JOIN


LEFT JOIN
Left Table
Right Table
RIGHT JOIN
Left Table
Right Table
INNER JOIN
Left Table
Right Table
FULL JOIN
Left Table
Right Table

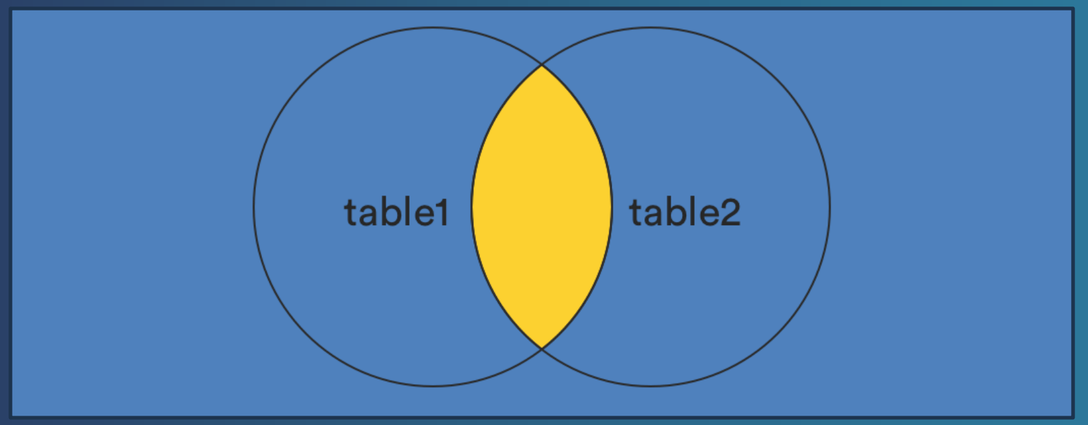
JOIN

```sql
SELECT columns_from_both_tables
FROM table1
JOIN table2
ON table1.column1 = table2.column2
```



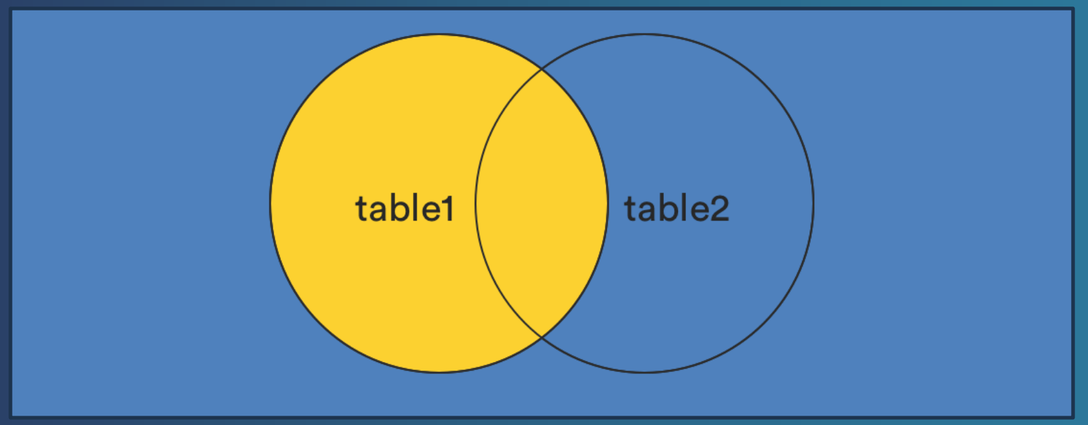
INNER JOIN

```
SELECT columns_from_both_tables
FROM table1
INNER JOIN table2
ON table1.column1 = table2.column2
```



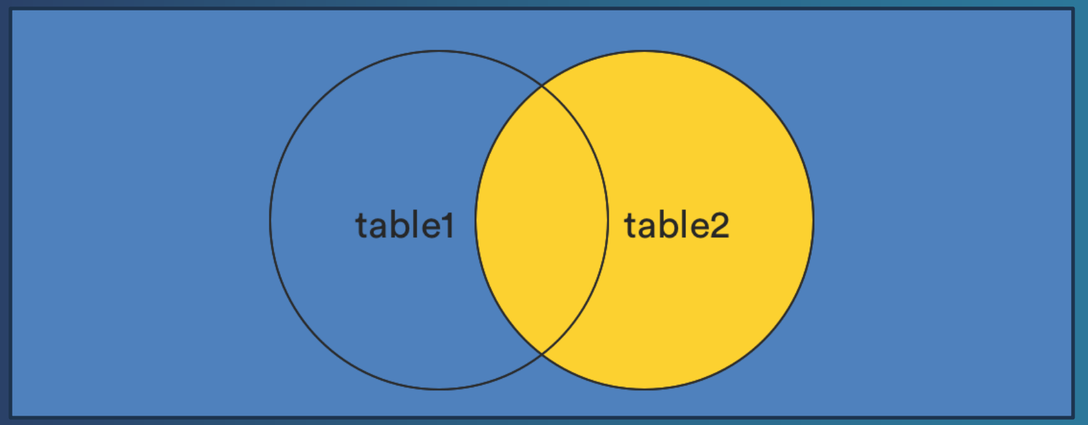
LEFT JOIN

```sql
SELECT columns_from_both_tables
FROM table1
LEFT JOIN table2
ON table1.column1 = table2.column2
```



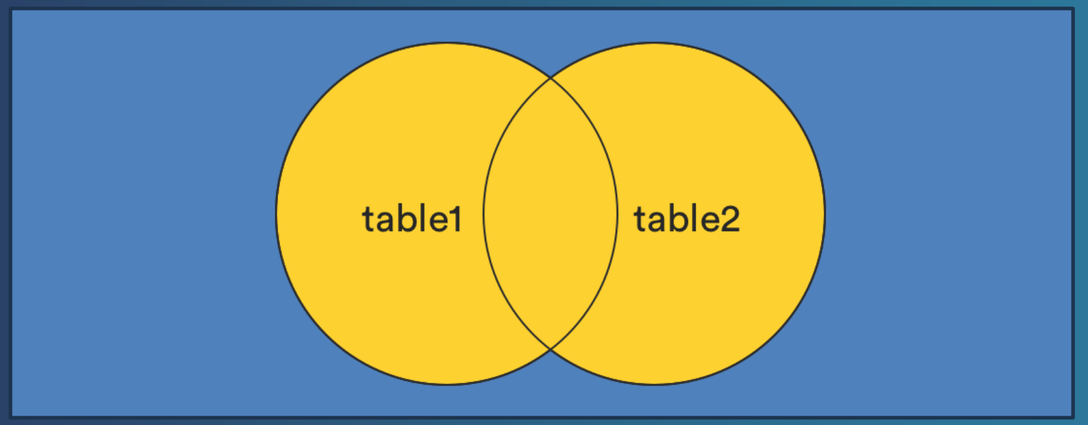
RIGHT JOIN

```
SELECT columns_from_both_tables
FROM table1
RIGHT JOIN table2
ON table1.column1 = table2.column2
```



FULL JOIN

```
SELECT columns_from_both_tables
FROM table1
FULL JOIN table2
ON table1.column1 = table2.column2
```





Komanda INSERT INTO

Komanda INSERT INTO përdoret për të shtuar rreshta të reja në tabele.

- Sintaksa

```
INSERT INTO emri_tabeles
VALUES (vlera1, vlera2,....)
```

Ju gjithsesi mund të tregoni se ku dëshironi të vendosni vlerat:

```
INSERT INTO emri_tabeles(kolona1, kolona2,...)
VALUES (vlera1, vlera2,....
```

Insertimi i një rreshti të ri

Ne tabelën "Personat" :

PersonID	Mbiemri	Emri	Email	Qyteti
111	Jakupi	Samir	Samir.jakupi@universitetiaab.com	Prishtinë

Nese e shënojmë komanden si më poshtë :

```
INSERT INTO Personat

VALUES (112,'Qarkaxhija','Jusuf','jusuf.qarkaxhija@universitetiaab.com','Gjakovë')
```

PersonID	Mbiemri	Emri	Email	Qyteti
111	Jakupi	Samir	Samir.jakupi@universitetiaab.com	Prishtinë
112	Qarkaxhija	Jusuf	jusuf.qarkaxhija@universitetiaab.com	Gjakovë

Insertimi i të dhënave në kolonë të caktuar

Ne tabelën "Personat" :

PersonID	Mbiemri	Emri	Email	Qyteti
111	Jakupi	Samir	Samir.jakupi@universitetiaab.com	Prishtinë
112	Qarkaxhija	Jusuf	jusuf.qarkaxhija@universitetiaab.com	Gjakovë

Nese e shënojmë komanden si më poshtë :

```
INSERT INTO Personat (Mbiemri, Email)
VALUES ('Berisha', 'leron.berisha@universitetiaab.com')
```

PersonID	Mbiemri	Emri	Email	Qyteti
111	Jakupi	Samir	Samir.jakupi@universitetiaab.com	Prishtinë
112	Qarkaxhija	Jusuf	jusuf.qarkaxhija@universitetiaab.com	Gjakovë
	Berisha		leron.berisha@universitetiaab.com	



Komanda UPDATE

Komanda UPDATE përdoret për të modifikuar shënimet apo të dhënat e caktuara.

- Sintaksa

```
UPDATE emri_tabelës
SET emri_kolones = vlera_re
WHERE emri_kolones = vlera_caktuar
```

Modifikimi i të dhënave

Nëse e shohim tabelën e mëposhtme Personat dhe deshirojmë të vendosim emrin aty ku gjindet mbiemri ‘Berisha’ kemi:

PersonID	Mbiemri	Emri	Email	Qyteti
111	Jakupi	Samir	Samir.jakupi@universitetiaab.com	Prishtinë
112	Qarkaxhija	Jusuf	jusuf.qarkaxhija@universitetiaab.com	Gjakovë
	Berisha		leron.berisha@universitetiaab.com	

```
UPDATE Personat SET Emri = 'Leron' WHERE Mbiemri = 'Berisha'
```

PersonID	Mbiemri	Emri	Email	Qyteti
111	Jakupi	Samir	Samir.jakupi@universitetiaab.com	Prishtinë
112	Qarkaxhija	Jusuf	jusuf.qarkaxhija@universitetiaab.com	Gjakovë
	Berisha	Leron	leron.berisha@universitetiaab.com	

Modifikimi i disa kolonave njëkohësisht

PersonID	Mbiemri	Emri	Email	Qyteti
111	Jakupi	Samir	Samir.jakupi@universitetiaab.com	Prishtinë
112	Qarkaxhija	Jusuf	jusuf.qarkaxhija@universitetiaab.com	Gjakovë
	Berisha	Leron	leron.berisha@universitetiaab.com	

Nëse dëshirojmë të shtojmë emrin e qytetit dhe të ndryshojmë email:

```
UPDATE Personat
SET Qyteti = 'Pejë', Email = 'leron1.berisha@universitetiaab.com'
WHERE Mbiemri = 'Berisha'
```

PersonID	Mbiemri	Emri	Email	Qyteti
111	Jakupi	Samir	Samir.jakupi@universitetiaab.com	Prishtinë
112	Qarkaxhija	Jusuf	jusuf.qarkaxhija@universitetiaab.com	Gjakovë
	Berisha	Leron	leron.berisha@universitetiaab.com	

Modifikimi i të dhënave

Nëse e modifikojmë tabelën pa vendosur kusht atëherë kemi:

PersonID	Mbiemri	Emri	Email	Qyteti
111	Jakupi	Samir	Samir.jakupi@universitetiaab.com	Prishtinë
112	Qarkaxhija	Jusuf	jusuf.qarkaxhija@universitetiaab.com	Gjakovë
	Berisha		leron.berisha@universitetiaab.com	

```
UPDATE Personat SET Emri = 'Leron'
```

PersonID	Mbiemri	Emri	Email	Qyteti
111	Jakupi	Leron	Samir.jakupi@universitetiaab.com	Prishtinë
112	Qarkaxhija	Leron	jusuf.qarkaxhija@universitetiaab.com	Gjakovë
	Berisha	Leron	leron.berisha@universitetiaab.com	



Komanda DELETE

Komanda Delete përdoret për të fshirë rreshtin:

- Sintaksa

```
DELETE FROM emri_tabeles
WHERE emri_kolones = vlera_caktuar
```

Fshirja e një rreshti

PersonID	Mbiemri	Emri	Email	Qyteti
111	Jakupi	Samir	Samir.jakupi@universitetiaab.com	Prishtinë
112	Qarkaxhija	Jusuf	jusuf.qarkaxhija@universitetiaab.com	Gjakovë
	Berisha	Leron	leron.berisha@universitetiaab.com	

Nëse dëshirojmë të fshijmë personin me emrin Leron, kemi:

```
DELETE FROM  Personat

WHERE Mbiemri = ‘Berisha'
```

PersonID	Mbiemri	Emri	Email	Qyteti
111	Jakupi	Samir	Samir.jakupi@universitetiaab.com	Prishtinë
112	Qarkaxhija	Jusuf	jusuf.qarkaxhija@universitetiaab.com	Gjakovë



Fshirja e të gjithë rreshtave në tabelë

Është e mundur që të fshihen të gjitha rreshtat pa e fshirë tabelen fare. Kjo do të thotë që atributet, indekset, struktura do të jetë e pa prekur, duke përdorur komandën në vijim:

```
DELETE FROM emri_tabeles
```



Krijimi i bazës së të dhënave

Për të krijuar bazat e të dhënave përdorim si më poshtë:

```
CREATE DATABASE emri_bazes_se_te_dhenave
```

Krijimi i tabeles

Për të krijuar tabelë përdorim si më poshtë:

```
CREATE TABLE emri_tabeles
(
 kolona1 tipi_i_te_dhenave,
Kolona2 tipi_i_te_dhenave, .....
 )
```



SHEMBULL

Ky shembull demonstron se si krijohet tabela me emrin "Personat", e cila ka katër kolona.
Emrat e kolonave do të jenë "Mbiemri", "Emri", "Adresa", dhe "Mosha”.

```
CREATE TABLE Personat (
PersonID int,
Mbiemri nvarchar,
Emri nvarchar,
Adresa nvarchar,
Mosha int
)
```



SHEMBULL

Kurse ky shembull tregon si mund të përcaktoni gjatësinë maksimale për secilen fushë,

```
CREATE TABLE Personat (
PersonID int,
Mbiemri nvarchar(30),
Emri nvarchar(20),
Adresa nvarchar(40),
Mosha int
)
```



Krijimi i indekseve

- Indekset krijohen në tabelë për arsye të gjinden rreshtat në tabelë shumë më shpejtë.
- Është e mundshme të krijohen indekset në një ose më shumë kolona të tabelës dhe secili indeks mund të ketë emrin e vet.
- Ata që i përdorin nuk mund t'i shohin indekset por mund t'I përdorin për t'a shpejtuar punën gjatë punës me queries.
- Vërejtje: Ndryshimi i shënimeve në tabelë me indeksa merr kohë me shumë se sa tabela pa indeksa, kjo është për arsyen se ndërrimi bëhet në të gjitha indekset. Kështu, të krijohen indekset vetëm në ato kolona në të cilat bëhen kërkesa për kritere.



Indeksi UNIK

Krijimi i indekseve unike.
Indeksi unik nënkupton që dy rreshta nuk mund të kenë vlerë të njejtë.

CREATE UNIQUE INDEX emri_indeksit **ON** emri_tabeles (emri_kolones)

"emri_kolones" tregon se cilën kolonë dëshironi t'a indeksoni.



DROP

Ju mund te fshini indekset përmes komandës DROP.

DROP INDEX emri_tabeles.emri_indeksit

Fshirja e bazës së të dhënave ose tabelës

Të fshihet baza e të dhënave përdorim si më poshtë:

DROP DATABASE emri_bazes_te_dhenave

Te fshihet tabela (struktura,atributete dhe indekset gjithashtu do të fshihen):

DROP TABLE emri_tabeles



Ndryshimi i tabelës

Komanda **ALTER TABLE** përdoret për shtimin apo largimin e kolonave nga tabela.

Shtimi i një kolone:

```
ALTER TABLE emri_tabeles
ADD emri_kolones tipi_i_te_dhenave
```

Largimi i një kolone:

```
ALTER TABLE emri_tabeles
DROP COLUMN emri_kolones
```



SHEMBULL

Tabela Personat.

PersonID	Mbiemri	Emri	Email	Qyteti
111	Jakupi	Samir	Samir.jakupi@universitetiaab.com	Prishtinë

Të shtohet kolona me emrin **NumriPersonal** në tabelën **Personat**.

```
ALTER TABLE Personat
ADD NumriPersonal nvarchar(15)
```

PersonID	Mbiemri	Emri	Email	Qyteti	NumriPersonal
111	Jakupi	Samir	Samir.jakupi@universitetiaab.com	Prishtinë	

SHEMBULL

Tabela Personat.

PersonID	Mbiemri	Emri	Email	Qyteti	NumriPersonal
111	Jakupi	Samir	Samir.jakupi@universitetiaab.com	Prishtinë	

Të largohet kolona me emrin **NumriPersonal** në tabelën **Personat**.

```
ALTER TABLE Personat
DROP COLUMN  Qyteti
```

PersonID	Mbiemri	Emri	Email	NumriPersonal
111	Jakupi	Samir	Samir.jakupi@universitetiaab.com	

Renditja e të dhënave

- Renditja e rreshtave

Klauzula **ORDER BY** perdoret per rënditjen ose sortimin e tabeles apo me saktesisht të kolones.

Shembull,

Kompanite	NumriPorosise
BRK	3412
Ardi	5678
IT Department	2312
IT Department	6798

Nëse dëshirojmë ti paraqesim kompanitë e renditura sipas alfabetit:

```
SELECT Kompanite, NumriPorosise
FROM Porosite
ORDER BY Kompanite
```

Kompanite	NumriPorosise
Ardi	5678
BRK	3412
IT Department	2312
IT Department	6798



SHEMBULL

Të paraqiten kompanitë sipas renditjes alfabetike dhe numrit të porosive në renditje numerike.

```
SELECT Kompanite, NumriPorosise
FROM Porosite
ORDER BY Kompanite, NumriPorosise
```

Kompanite	NumriPorosise
Ardi	5678
BRK	3412
IT Department	2312
IT Department	6798



SHEMBULL

Të paraqiten kompanitë sipas renditjes alfabetike dhe numrit të porosive në renditje numerike.

```
SELECT Kompanite, NumriPorosise
FROM Porosite
ORDER BY Kompanite DESC
```

Kompanite	NumriPorosise
IT Department	6798
IT Department	2312
BRK	3412
Ardi	5678



SHEMBULL

Të paraqiten kompanitë sipas renditjes alfabetike dhe numrit të porosive në renditje numerike.

```
SELECT Kompanite, NumriPorosise
FROM Porosite
ORDER BY Kompanite DESC, NumriPorosise ASC
```

Kompanite	NumriPorosise
IT Department	2312
IT Department	6798
BRK	3412
Ardi	5678



GROUP BY

GROUP BY është shtuar në SQL sepse funksionet aggregate (p.sh. SUM) kthejnë vlera aggregate nga kolona sa here që të thirrën, dhe pa funksionin **GROUP BY** është e pamundur të gjendet shuma për secilin grup individual për vlerat e kolonës.

Sintaksa e funksionit **GROUP BY** është:

SELECT emri_kolones, **SUM**(emri_kolones)
FROM emri_tabeles
GROUP BY emri_kolones



SHEMBULL

Tabela Shitjet.

Kompanite	Sasia
IT Department	5500
Ardi	4500
IT Department	7100

```
SELECT Kompanite, SUM(sasia)
FROM shitjet
```

Kompanite	SUM(Sasia)
IT Department	17100
Ardi	17100
IT Department	17100



SHEMBULL

Rezultati paraprak është i pakuptueshëm dhe nuk është me pjesë e funksioneve aggregate.

Prandaj duhet të përdoret GROUP BY për të zgjidhur këtë problem.

```
SELECT Kompanite, SUM(sasia)
FROM shitjet
GROUP BY Kompanite
```

Kompanite	SUM(Sasia)
IT Department	12600
Ardi	4500



HAVING

HAVING i është shtuar SQL sepse fjala e rezervuar WHERE nuk përdoret gjatë thirrjeve të funskioneve aggregate (p.sh. SUM), si dhe pa fjalën e rezervuar HAVING është e pamundur të thirren kushtet për rezultat.

Sintaksa e funksionit **HAVING** është:

```
SELECT emri_kolones, SUM(emri_kolones)
FROM emri_tabeles
GROUP BY emri_kolones
HAVING SUM(emri_kolones) vlera_e_kushtit
```



SHEMBULL

Tabela Shitjet.

Kompanite	Sasia
IT Department	5500
Ardi	4500
IT Department	7100

SELECT Kompanite, SUM(sasia)

FROM shitjet

GROUP BY Kompanite

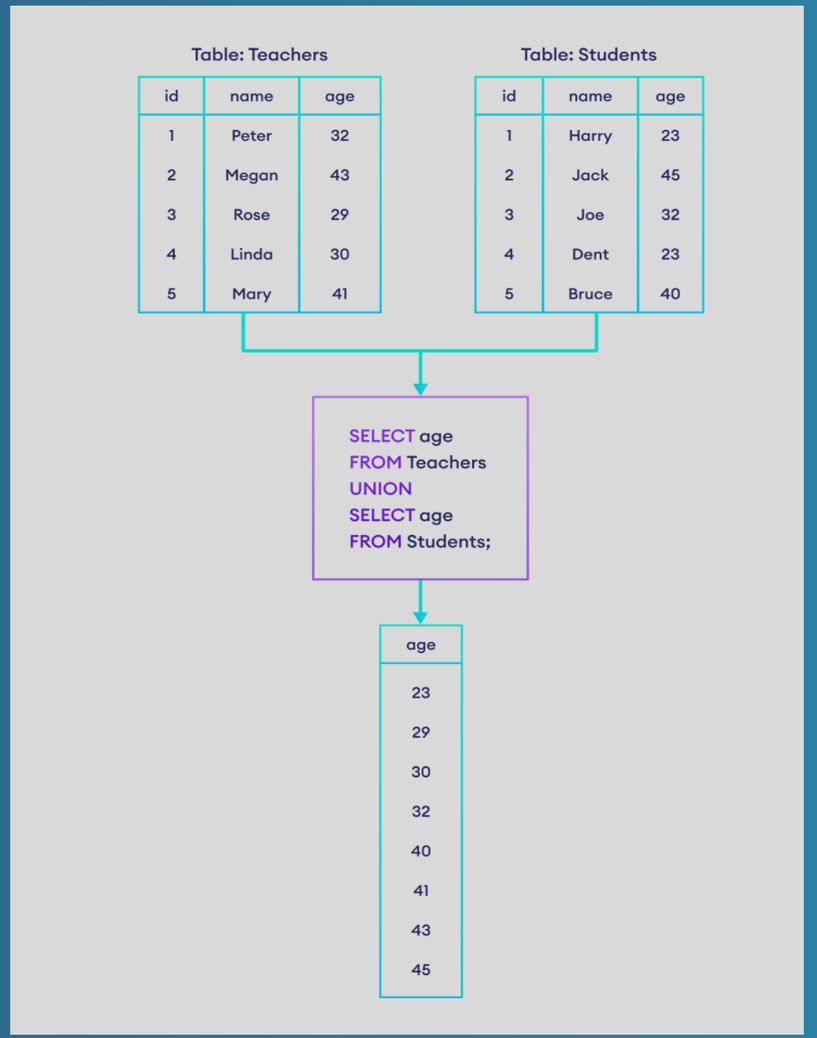
HAVING SUM(sasia)>10000

Kompanite	SUM(Sasia)
IT Department	12600

UNION

- Në SQL, operatori UNION zgjedh fushat nga dy ose më shumë tabela.
- Sintaksa e funksionit UNION është:

```
SELECT column1, column2, ...
FROM table1
UNION
SELECT column1, column2, ...
FROM table2;
```

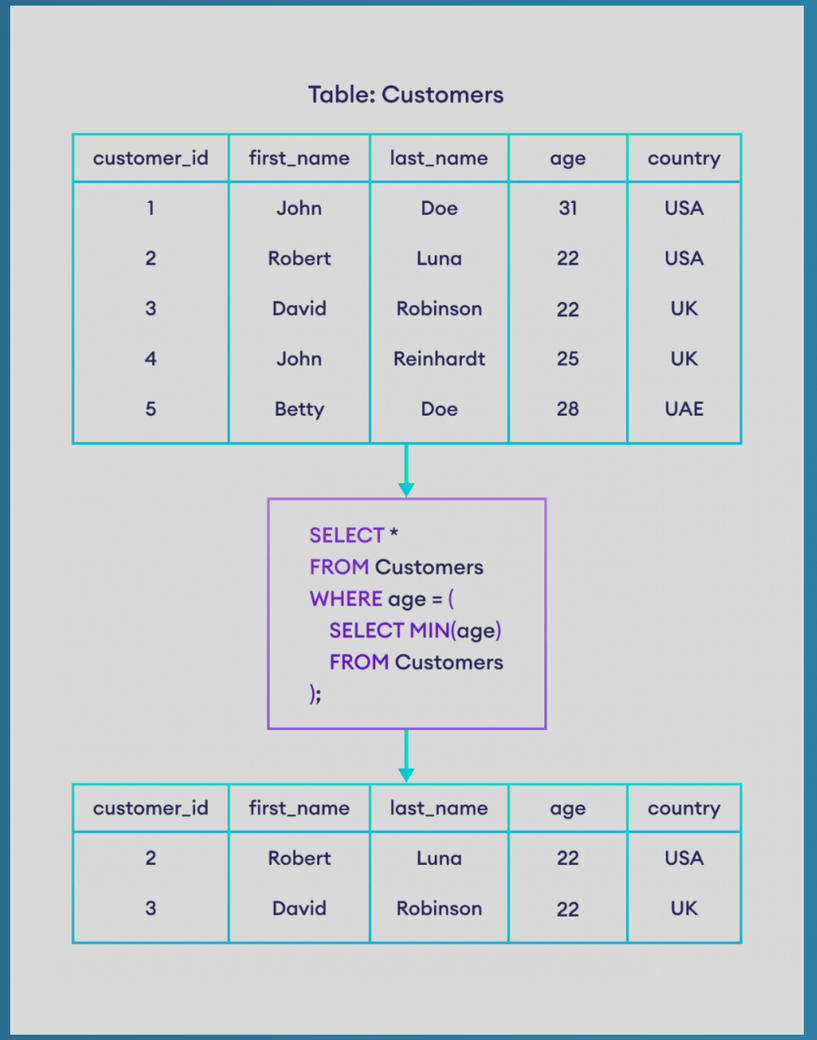


SUBQUERY

Në SQL, është e mundur të vendosni një query SQL brenda një Query tjetër. Kjo query e brendshme njihet si subquery.

Sintaksa e funksionit SUBQUERY është:

```
SELECT column
 FROM table
WHERE column OPERATOR ( SELECT column FROM table )
```



JU FALEMINDERIT!